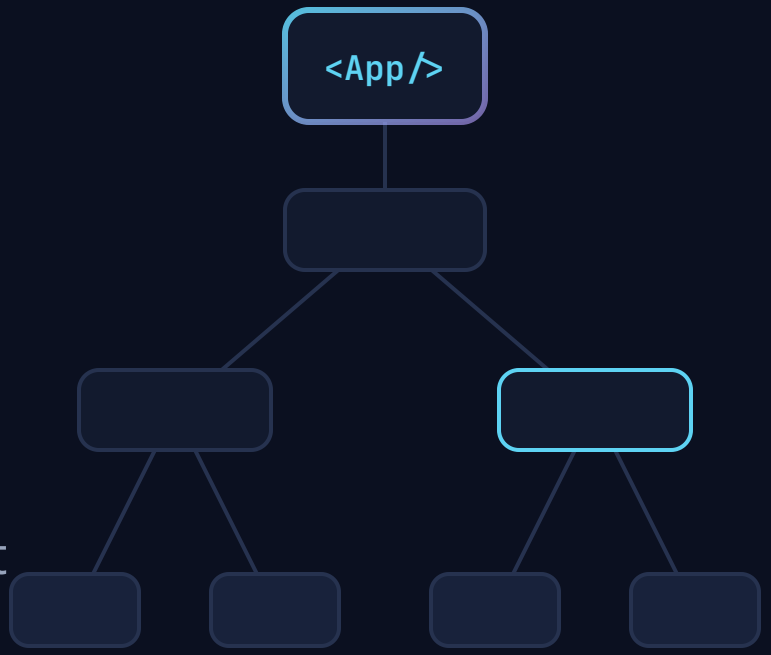
ვებტექნოლოგიები · ლექცია 1

React-ის შესავალი

რატომ გაჩნდა front-end ფრეიმვორკები, რა არის React და როგორ ავაწყოთ პირველი პროექტი 2026 წელს



წყარო: **The Ultimate React Course** ვერსია: **React 19.3**

დღის გეგმა

რას გავივლით დღეს

1

რატომ ფრეიმვორკი?

SSR, SPA და UI-ს სინქრონი მონაცემებთან

2

Vanilla JS vs React

ერთი და იგივე ამოცანა — ორი მიდგომა

3

რა არის React

კომპონენტები, JSX, state, ეკოსისტემა

4

Pure React

React ერთ HTML ფაილში, build-ის გარეშე

5

პირველი პროექტი

Vite, საქაღალდეების სტრუქტურა, react.dev

6

შეჯამება

კითხვები და საშინაო დავალება

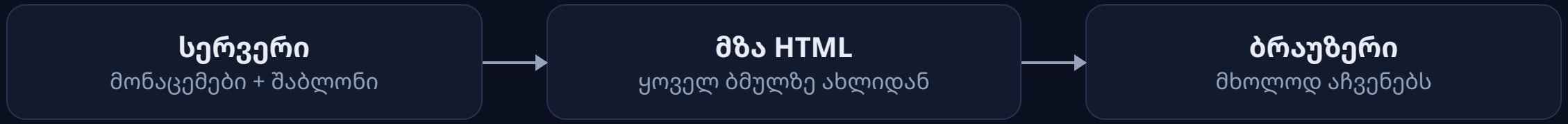
01

რატომ არსებობს front-end ფრეიმვორკები?

სერვერი თუ ბრაუზერი?

ვინ აწყობს გვერდს

ძველი გზა · SERVER-SIDE RENDERING



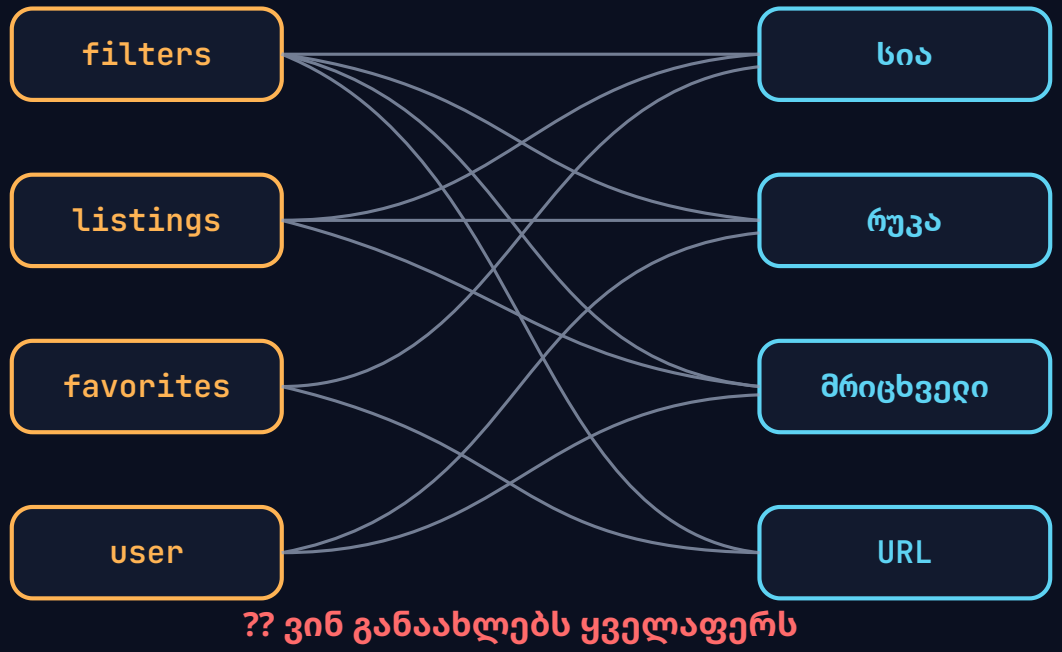
ახალი გზა · CLIENT-SIDE RENDERING / SPA



SPA — Single-Page Application: გვერდი არ იტვირთება ხელახლა და აპი დესკტოპური პროგრამასავით იქცევა.

მთავარი პრობლემა

UI უნდა ემთხვეოდეს მონაცემებს. ყოველთვის.



აპი = **მონაცემების მართვა** + **მათი ჩვენება**.

ერთი ცვლილება ეკრანზე **რამდენიმე ადგილს** ეხება.

> მონაცემის ერთ ნაწილს ვეძახით **state**-ს (მდგომარეობა).

VANILLA JAVASCRIPT

რატომ არის ეს რთული უბრალო JS-ით

პრობლემა 1

ბევრი პირდაპირი DOM მანიპულაცია

querySelector → textContent → appendChild... ყოველ ცვლილებაზე ხელით. შედეგი — „სპაგეტი კოდი“.

პრობლემა 2

state DOM-შია და ყველგანაა

მონაცემები ელემენტებში და გლობალურ ცვლადებშია მიმოფანტული. ძნელია გააზრება, ბაგები ბევრია.

```
// ერთი ფილტრის ცვლილება ...
list.innerHTML = "";
results.forEach(r ⇒ list.appendChild(makeCard(r)));
counter.textContent = results.length + " ბინა";
map.clearMarkers(); results.forEach(r ⇒ map.addMarker(r));
history.pushState({}, "", "?price=" + price);
```

გამოსავალი

ფრეიმვორკი ამ სამუშაოს თავზე იღებს

1

სინქრონი

UI-ს მონაცემებთან თვითონ ათანხმებს — DOM-ს ხელით აღარ ვეხებით

2

სტრუქტურა

კოდის „სწორ“ წყობას გვიკარნახებს — სპაგეტი კოდის წინააღმდეგ

3

ერთიანი წესები

გუნდი ერთნაირად წერს აპლიკაციებს

სხვადასხვა მიდგომა — ერთი მიზანი:

React | Angular | Vue | Svelte

იმპერატიული VS დეკლარაციული

ერთი მთვლელი — ორი მიდგომა

Vanilla JS ვწერთ, **როგორ** შეიცვალოს

```
let count = 0;
const span = document.querySelector("#count");
const btn = document.querySelector("#btn");

btn.addEventListener("click", () ⇒ {
  count++;
  span.textContent = count; // DOM-ს ვეხებით
});
```

React ვწერთ, **როგორი** უნდა იყოს

```
function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() ⇒ setCount(count + 1)}>
      დაკლიკება: {count}
    </button>
  );
}
```

> React-ში ვცვლით მხოლოდ **state**-ს — ეკრანს React თვითონ განაახლებს.

ცოცხალი დემო

state იცვლება → UI თავიდან იხატება

state

```
state = {
  city: "ყვედა",
  cheapOnly: false
}
```

ყველა | თბილისი | ბათუმი | ქუთაისი

მხოლოდ ≤ 100 ₾

UI = render(state)

5 ბინა · ყველა

ბინა ვაკეში თბილისი	140 ₾
სტუდიო სოლოლაკში თბილისი	85 ₾
ბინა ზღვასთან ბათუმი	120 ₾
ოთახი ძველ ბათუმში ბათუმი	60 ₾
სახლი ცენტრში ქუთაისი	70 ₾

render() გამოძახება: 1

02

რა არის React?

განმარტება

React არის JavaScript ბიბლიოთეკა მომხმარებლის ინტერფეისების ასაწყობად

სახელი შემთხვევითი არ არის: როცა state იცვლება, React **რეაგირებს** და UI-ს თავიდან არენდერებს.

1. Render

კომპონენტებს აჩვენებს მიმდინარე state-ის მიხედვით

2. Re-render

UI-ს state-თან სინქრონში ინახავს

6 მახასიათებელი

რით გამოირჩევა React

🧱 კომპონენტებზე

UI იკრიბება პატარა, ხელახლა გამოყენებადი ნაწილებისგან

📝 დეკლარაციული

ვაღწერთ, როგორი უნდა იყოს UI — DOM-ს არ ვეხებით

🔁 State-driven

state იცვლება → React UI-ს თავიდან ხატავს

⟨/⟩ JSX

HTML-ის მსგავსი სინტაქსი JavaScript-ის შიგნით

📚 ბიბლიოთეკა

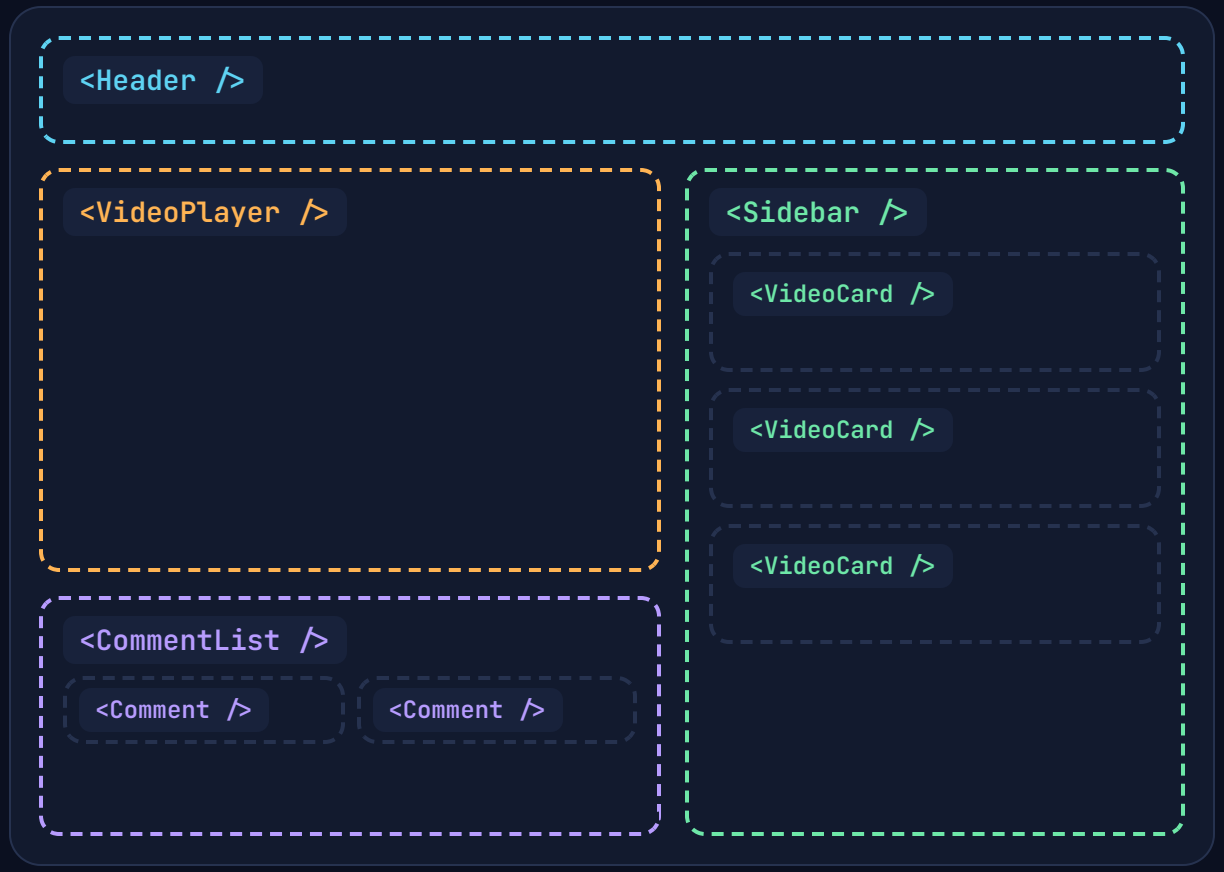
მხოლოდ „view“ შრე — დანარჩენს ჩვენ ვირჩევთ

🌍 პოპულარული

უდიდესი საზოგადოება, ეკოსისტემა და სამუშაო ბაზარი

კომპონენტები

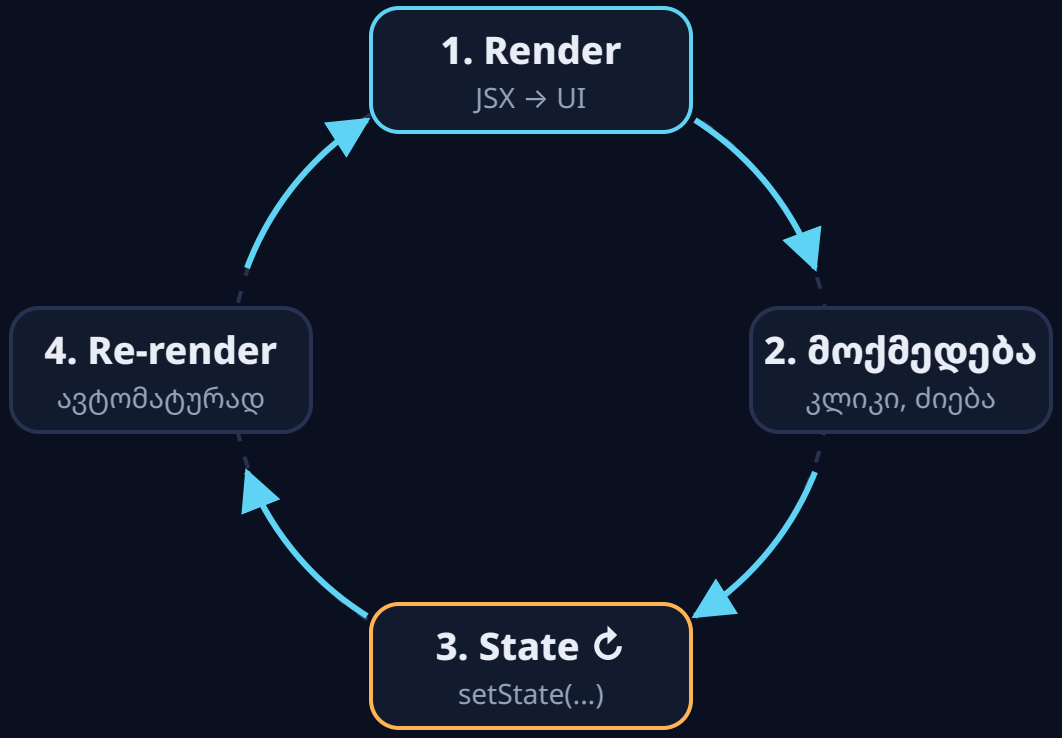
UI = კომპონენტების „ლეგო“



- კომპონენტი = **მონაცემი + ლოგიკა + გარეგნობა**
- კომპონენტები **ერთმანეთში ჩაიდგმება** და ქმნის ხეს
- ერთი კომპონენტი **მრავალჯერ** გამოიყენება (VideoCard, Comment)
- მონაცემი მშობლიდან შვილს **props**-ით გადაეცემა

STATE-DRIVEN

React რეაგირებს state-ის ცვლილებაზე



ჩვენ ვმართავთ **მხოლოდ state-ს**.

React თვითონ წყვეტს, **რა** შეიცვალა ეკრანზე და **როგორ** განაახლოს.

> ამიტომ ჰქვია **React** — ის „რეაგირებს“.

ბიბლიოთეკა თუ ფრეიმვორკი?

React მხოლოდ „view“ შრეა

ეკოსისტემა — ვირჩევთ თვითონ

რაც React-ში არ არის, ცალკე ბიბლიოთეკით ემატება:

- React Router
- TanStack Query
- Redux Toolkit
- Tailwind CSS
- Styled Components

React-ის ფრეიმვორკები

სრული გადაწყვეტა რეალური პროდუქტებისთვის:

- Next.js
- React Router (framework)
- Expo (mobile)

> სასწავლად ჯერ ვსწავლობთ **სუფთა React-ს** — ფრეიმვორკი მის ცოდნას მოითხოვს.

ისტორია

15 წელი React-ის ცხოვრებაში

2011
Jordan Walke ქმნის Facebook-ში

2013
ღია კოდი (open source)

2015
React Native — მობილური აპები

2019
Hooks (16.8) — useState, useEffect

2024
React 19 (დეკემბერი)

2025
Create React App მოძველდა

2026
React Foundation (Linux Foundation)

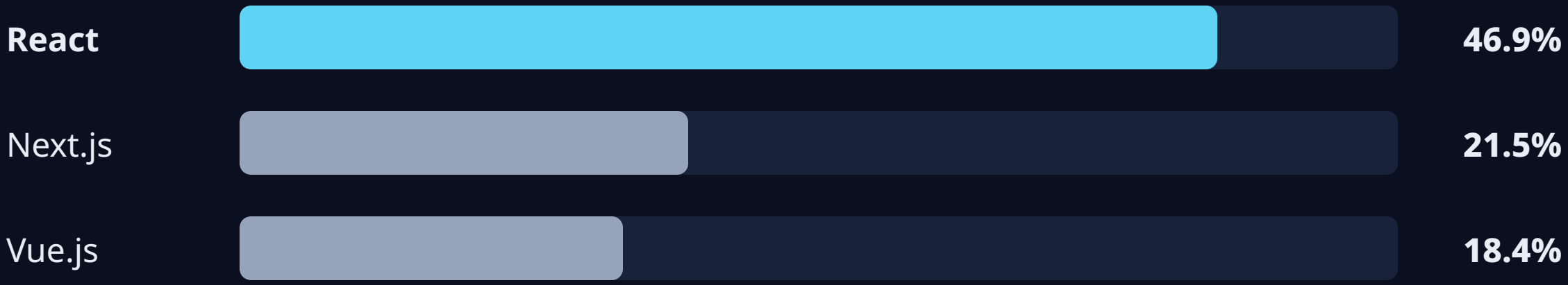
დღეს
React 19.3
9 სექტ. 2026

> 2019 წლის **Hooks** (useState, useEffect) — სწორედ მათ გამოვიყენებთ მთელ კურსში.

პოპულარობა

ყველაზე გამოყენებადი ვებ ტექნოლოგია

Stack Overflow Developer Survey 2025 · დეველოპერების წილი, ვინც იყენებს



Next.js თავადაც **React-ზეა** აშენებული — React-ის ცოდნა ორივეს გიხსნის.

03

დავიწყოთ კოლი

PURE REACT

React ერთ HTML ფაილში — build-ის გარეშე

```
<div id="root"></div>
<script type="module">
  import React from "https://esm.sh/react@19/?dev";
  import ReactDOM from "https://esm.sh/react-dom@19/client?dev";

  const now = () ⇒ new Date().toLocaleTimeString();

  function App() {
    const [time, setTime] = React.useState(now());
    React.useEffect(() ⇒ {
      const id = setInterval(() ⇒ setTime(now()), 1000);
      return () ⇒ clearInterval(id);
    }, []);
    return React.createElement("h1", null, "ახლა: " + time);
  }

  ReactDOM.createRoot(document.getElementById("root"))
    .render(React.createElement(App));
</script>
```

- **App** — ჩვეულებრივი ფუნქცია = კომპონენტი
- **useState** ინახავს დროს, ყოველ წამში ახლდება
- **createRoot().render()** React-ს #root-ზე „ამაგრებს“

> **2026:** React 19-დან UMD სკრიპტები აღარ გამოდის — ვიყენებთ `esm.sh`-ს

პროექტის შექმნა 2026-ში

რით შევქმნათ React პროექტი

ინსტრუმენტი	სტატუსი	როდის
Create React App	მოძველებული · 2025	მხოლოდ ძველ პროექტებსა და გაკვეთილებში შეხვდებით
Vite	ვიყენებთ ამ კურსში	სწავლა, ექსპერიმენტები, SPA. სწრაფი dev სერვერი და HMR
Next.js / React Router	ფრეიმვორკი	რეალური პროდუქტები: როუტინგი, SSR, მონაცემები

წყარო: react.dev/blog/2025/02/14/sunsetting-create-react-app

პრაქტიკა

პირველი პროექტი Vite-ით

ტერმინალი · Node.js 20.19+ / 22.12+

```
node -v
npm create vite@latest my-first-react -- --template react
cd my-first-react
npm install
npm run dev      # → http://localhost:5173
```

> შეცვალეთ ტექსტი `App.jsx`-ში და შეინახეთ — გვერდი მყისიერად განახლდება (**HMR**).

```
my-first-react/
├─ public/
├─ src/
│  ├─ App.jsx ← მთავარი კომპონენტი
│  ├─ App.css
│  ├─ main.jsx ← createRoot
│  └─ index.css
├─ index.html ← <div id="root">
├─ package.json
└─ vite.config.js
```

დოკუმენტაცია

react.dev — თქვენი მთავარი წყარო

LEARN

სახელმძღვანელო

Quick Start, Thinking in React, ინტერაქტიული მაგალითები

REFERENCE

API

ყველა hook და კომპონენტი: useState, useEffect...

BLOG

სიახლეები

ახალი ვერსიები და ცვლილებები

> რჩევა: დოკუმენტაცია სწავლის დროს **ყოველთვის გახსნილი** გქონდეთ. ძველი `reactjs.org` აღარ გამოიყენება.

შეჯამება

React-ის ორი მთავარი საქმე

1

Render

კომპონენტებს აჩვენებს მიმდინარე **state**-ის მიხედვით

2

Re-render

UI-ს სინქრონში ინახავს — **რეაგირებს**, როცა state იცვლება

დღეს ვისწავლეთ:

SPA | იმპერატიული vs დეკლარაციული | კომპონენტები | JSX | state | esm.sh | Vite

კითხვები და დავალება

შევამოწმოთ ცოდნა

კითხვები

1. რა პრობლემას ჭრის ფრეიმვორკი?
2. რით განსხვავდება SPA server-rendered საიტისგან?
3. რას ნიშნავს „დეკლარაციული“?
4. React ბიბლიოთეკაა თუ ფრეიმვორკი? რატომ?
5. რატომ აღარ ვიყენებთ Create React App-ს?

საშინაო დავალება

- შექმენით Vite + React პროექტი
- Pure React საათს დაუმატეთ თარიღი
- დახატეთ ერთი საიტის კომპონენტების ხე (მინ. 6)
- წაიკითხეთ react.dev → Quick Start